Preparcial: Cloud Center

El departamente de IT de un empresa recibe un CSV con información referida a la telemetria de sus servidores. El programa carga los datos de los servidores y el costo relacionado con esos estados. Este proyecto se entrega para estudiar y ejecutar; no hay tareas de implementación para esta instancia.

Ejecución

Requiere JDK 21 o 25 y Maven 3.9.x. Desde esta carpeta:

```
mvn test
java -cp target/classes ar.edu.backend.App
java -cp target/classes ar.edu.backend.App datos/datos.csv
```

La primera ejecución de Maven puede requerir Internet. Una vez resueltas las dependencias, probar `mvn -o test` en la misma computadora. El programa no requiere dependencias externas en ejecución.

Contrato del archivo

UTF-8, encabezado exacto `id,region,porcentajeCPU,usoMemoria,estado`. Cada registro ocupa una línea con cinco campos separados por coma. Los campos no admiten comas, comillas ni saltos de línea internos; no hay campos entrecomillados. Se quitan espacios externos de cada campo. Es un formato de intercambio controlado, no un lector de todo CSV posible. El uso de `split(",", -1)` conserva los campos finales vacíos y es válido bajo este contrato.

Cada fila tiene un identificado y region no vacios, porcentaje de cpu y cantidad de memoria en GB valores entero que representan uso, y el estado en el que se encuentra el servidor. No se exige unicidad de identificadores; cada fila válida cuenta como un servidor independiente. La region son textos sensibles a mayúsculas.

El porcentaje de uso del CPU debe ser un valor mayor a 0 y menor a 100, el uso de memoria no puede ser un valor mayor a 24576 GB. El indice de carga que se debe calcular es un valor entre 1 y 70. Estos cálculos y validaciones quedan cerca del objeto, en `ServidorEnvio.desdeCampos` y su constructor. El constructor recibe el **indiceCarga**.

El Costo del Riesto Operativo se calcula asumiendo que la aplicaciones que usan los servidores generan en promedio 1000 USD de ingresos. No se usan números decimales ni redondeos monetarios, será un valor entero.

Las formulas para resolver ambos valores es el siguiente:

- Indice de carga

$$indiceCarga = (porcentajeUsoCPU \times 0.7) + (memoriaConsumidaGB \times 0.3)$$

- costoRiesgoOperativo

$$\text{Costo del Riesgo Operativo} = 1000 \times \left(\frac{\text{Índice de Carga}}{100} \right)$$

Flujo y errores

El parser verifica primero la cantidad de columnas. Con ancho correcto, `ESTABLE` se descarta sin validar los restantes datos. `ALERTA` y `CRITICO` se intentan convertir; cualquier otro estado es inválido. Una línea vacía es inválida. No se interrumpe la carga por errores de una fila: se registra el número físico de línea y el motivo. Un encabezado incorrecto aborta con `IllegalArgumentException`; un fallo de lectura propaga `IOException`. El encabezado no cuenta como fila leída.

Recorrido de lectura sugerido

1. Ejecutar `App` y los tres grupos de tests.
2. Leer `Servidor`: invariantes, conversión textual y costo derivado.
3. Seguir `ParserServidores` y `ResultadoParseo`: clasificación y manejo de excepciones.
4. Leer `Cloud`: copia defensiva, `Predicate<Servidor>`, filtros, suma y agrupamiento.
5. Localizar casos testigo en el archivo y explicar su clasificación y costo; usar el programa para verificar los agregados del conjunto.

El archivo contiene 60 filas de datos más encabezado (61 líneas). Panorama esperado: 60 leídas, 30 procesadas, 21 descartadas y 9 inválidas; total 15170, por region SA-EAST-1 5400, US-EAST-1 6790, US-WEST-2 2980; 16 Servidores con un indice de carga menores a 50. Casos testigos: srv-0044 (50), srv-0021 (50), srv-0045 (descarte por estado ESTABLE), srv-0006 (invalido por indice de carga), srv-0004 (invalido estado desconocido).

Los getters se escriben explícitamente: no se agrega Lombok para minimizar dependencias y configuración del entorno de examen. ResultadoParseo también usa getters convencionales y conserva copias defensivas de sus listas. JUnit 5 es la única dependencia y se usa solamente en tests.